

SMART OFFICE 2.0

Modèle Conceptuel de Données — Merise

Application

room-booking — Réservation de salles de réunion

Équipe projet

Sam Flaujat-Ellul | Alexander Kedevil

Formation

Bachelor 3 — Spécialité Infrastructure & Réseaux / Cybersécurité

Établissement

Ynov Campus Montpellier — UF INFRA B3

Juin 2026

Introduction

Ce document présente la modélisation conceptuelle et logique de la base de données de l'application room-booking. Le modèle suit la méthode Merise et est aligné sur le schéma PostgreSQL implémenté.

La méthode Merise est la norme de modélisation de données dans les référentiels de formation informatique français. Elle structure la conception en trois niveaux (conceptuel → logique → physique), permettant de valider la logique métier avant l'implémentation SQL. La modélisation préalable a permis d'éviter plusieurs erreurs de conception, notamment sur les cardinalités de RESERVATION vis-à-vis de SALLE.

1. MCD — Modèle Conceptuel de Données

1.1 Entités

Entité	Attributs clés	Rôle
UTILISATEUR	id (PK), email , role	Compte qui réserve une salle
SALLE	id (PK), name , capacity , floor	Salle de réunion physique
RESERVATION	id (PK), start_time , end_time	Créneau horaire réservé

Trois entités suffisent pour couvrir le besoin métier : qui réserve (UTILISATEUR), quoi (SALLE) et quand (RESERVATION). L'entité RESERVATION est indépendante — pas une simple table de liaison — car elle possède ses propres attributs métier (start_time , end_time) et ses propres règles de gestion (non-chevauchement).

1.2 Associations et cardinalités

Association	Entité	Card.	Entité	Card.
RESERVER	UTILISATEUR	(0,N)	RESERVATION	(1,1)
CONCERNE	RESERVATION	(1,1)	SALLE	(0,N)

La cardinalité (0,N) côté UTILISATEUR signifie qu'un compte peut effectuer zéro ou plusieurs réservations ; la cardinalité (1,1) côté RESERVATION garantit que chaque réservation est rattachée à exactement un utilisateur. De même, une réservation concerne exactement une salle (1,1) — pas de réservation multi-salles dans le modèle actuel, extension possible — tandis qu'une salle peut faire l'objet de zéro ou plusieurs réservations (0,N).

1.3 Représentation schématique



2. MLD — Modèle Logique de Données

Traduction du MCD en modèle relationnel :

```
rooms      (id, name, capacity, floor)
users      (id, email, role)
bookings   (id, room_id → rooms.id, user_id → users.id, start_time, end_time)
```

La table `bookings` centralise les clés étrangères `room_id` et `user_id`, conformément à la traduction standard des associations Merise de type (0,N) — (1,1) : la clé migre du côté (1,1). Les `CASCADE ON DELETE` (suppression en cascade) garantissent la cohérence référentielle : supprimer un utilisateur supprime automatiquement ses réservations, sans laisser d'enregistrements orphelins.

3. Modèle Physique — Schéma SQL PostgreSQL

3.1 Table SALLE (rooms)

Attribut	Type SQL	Contrainte
id	SERIAL	PRIMARY KEY
name	VARCHAR(100)	NOT NULL
capacity	INT	NOT NULL, > 0
floor	INT	NOT NULL, >= 0

Le champ `floor` (étage) est inclus pour permettre des recherches de disponibilité par étage (utile dans un bâtiment de 4 étages). La contrainte `> 0` sur `capacity` empêche les insertions incohérentes (salle de 0 place).

3.2 Table UTILISATEUR (users)

Attribut	Type SQL	Contrainte
id	SERIAL	PRIMARY KEY
email	VARCHAR(255)	UNIQUE, NOT NULL
role	VARCHAR(20)	'employee' 'admin'

L'email est la clé naturelle de l'utilisateur (`UNIQUE + NOT NULL`). Il est utilisé comme identifiant dans les logs de réservation, ce qui facilite le débogage sans jointure avec un annuaire externe. Le champ `role` est un `VARCHAR` plutôt qu'un `ENUM PostgreSQL` pour permettre l'ajout de rôles futurs sans migration `DDL`.

Note Entra ID : l'attribut `entra_oid` est prévu pour lier le compte Microsoft (non requis en mode `AUTH_DISABLED`). Il permettrait de réconcilier les utilisateurs Entra ID avec les enregistrements PostgreSQL sans dépendre de l'email comme identifiant.

3.3 Table RESERVATION (bookings)

Attribut	Type SQL	Contrainte
id	SERIAL	PRIMARY KEY
room_id	INT	FK → rooms.id , ON DELETE CASCADE
user_id	INT	FK → users.id , ON DELETE CASCADE
start_time	TIMESTAMPTZ	NOT NULL
end_time	TIMESTAMPTZ	NOT NULL, > start_time

Le type `TIMESTAMPTZ` (Timestamp with Time Zone) est essentiel pour une startup avec des employés potentiellement dans plusieurs fuseaux horaires. PostgreSQL stocke toujours en UTC et convertit à l'affichage selon le fuseau de la session, évitant les bugs de chevauchement liés aux changements d'heure.

4. Règles métier et contraintes

- **Non-chevauchement des réservations** : vérification dans l'API Flask (requête SELECT avec `OVERLAPS`)

La contrainte de non-chevauchement n'est pas implémentée comme une contrainte SQL `EXCLUDE` (PostgreSQL le supporte mais nécessite l'extension `btree_gist`). Elle est gérée dans l'API pour rester portable sur d'autres SGBD. L'index `idx_bookings_room_time` accélère cette vérification.

- **Cascade de suppression** (ON DELETE CASCADE) sur `room_id` et `user_id`
- **Index** `idx_bookings_room_time` pour les requêtes de disponibilité

Cet index composé sur (`room_id` , `start_time` , `end_time`) optimise la requête de disponibilité la plus fréquente : « quels créneaux sont occupés pour cette salle à cette date ? ». Sans cet index, la requête effectuerait un scan complet de la table `bookings` à chaque vérification.

5. Cache NoSQL — Redis

Redis est utilisé comme couche de cache de second niveau pour les disponibilités. PostgreSQL reste la source de vérité.

Clé Redis	Contenu	TTL
availability:{room_id}:{date}	Créneaux occupés (JSON)	Court (invalidé à chaque réservation)

Le cache Redis est invalidé à chaque création ou suppression de réservation (écriture PostgreSQL → suppression de la clé Redis). Ce schéma cache-aside garantit la cohérence : le cache ne sera jamais plus récent que la BDD. Le TTL court est un filet de sécurité supplémentaire en cas de bug d'invalidation.